

Oracle Regular Expressions

REGEXP_LIKE, SUBSTR, INSTR, REPLACE, COUNT & pattern syntax · Oracle 19c

Oracle supports POSIX-style regular expressions through five functions. They enable powerful pattern matching, extraction, and replacement far beyond `LIKE`.

1. The Five REGEXP Functions

Function	Purpose	Returns
<code>REGEXP_LIKE(s,pat)</code>	Condition — does s match pat?	Boolean (WHERE clause)
<code>REGEXP_SUBSTR(s,pat)</code>	Extract the matching substring	The matched text
<code>REGEXP_INSTR(s,pat)</code>	Position where match starts	Number
<code>REGEXP_REPLACE(s,pat,r)</code>	Replace matches with r	New string
<code>REGEXP_COUNT(s,pat)</code>	Count matches	Number

2. Common Metacharacters

Pattern	Meaning	Example match
<code>.</code>	Any single character	<code>a.c</code> → abc, axc
<code>^ ... \$</code>	Start / end of string	<code>^A, z\$</code>
<code>* + ?</code>	0+, 1+, 0-or-1 repetitions	<code>ab+</code> → ab, abb
<code>{m,n}</code>	Between m and n times	<code>[0-9]{3,4}</code>
<code>[...]</code>	Character class (set)	<code>[A-Za-z]</code>
<code>[^...]</code>	Negated class	<code>[^0-9]</code> = non-digit
<code>()</code>	Group / capture	<code>(ab)+</code>
<code> </code>	Alternation (or)	<code>cat dog</code>
<code>\d \w \s</code>	Digit / word / whitespace	<code>\d+</code> = digits
<code>\</code>	Escape a metacharacter	<code>\.</code> = literal dot

POSIX character classes

Class	Class	Class
<code>[:alpha:]</code> letters	<code>[:digit:]</code> digits	<code>[:alnum:]</code> letters+digits
<code>[:space:]</code> whitespace	<code>[:upper:]</code> uppercase	<code>[:punct:]</code> punctuation

3. REGEXP_LIKE — matching in WHERE

```
-- rows whose code is exactly 4 digits
SELECT * FROM t WHERE REGEXP_LIKE(code, '[0-9]{4}$');

-- case-insensitive match ('i' flag)
SELECT * FROM t WHERE REGEXP_LIKE(name, '^ali', 'i');

-- validate an email
WHERE REGEXP_LIKE(email, '^([A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,})$');
```

4. REGEXP_SUBSTR — extraction

```
-- first number in a string
SELECT REGEXP_SUBSTR('Order #4521 shipped', '[0-9]+') FROM dual; -- 4521

-- nth occurrence & capture group: SUBSTR(s, pat, position, occurrence, flags, group)
SELECT REGEXP_SUBSTR('2026-08-07', '(\d{4})-(\d{2})-(\d{2})', 1, 1, NULL, 2)
FROM dual; -- '08' (2nd group)

-- split CSV into rows (nth element per row via CONNECT BY)
SELECT REGEXP_SUBSTR('a,b,c,d', '[^,]+', 1, LEVEL) part
FROM dual CONNECT BY REGEXP_SUBSTR('a,b,c,d', '[^,]+', 1, LEVEL) IS NOT NULL;
```

5. REGEXP_REPLACE — transform

```
-- remove all non-alphanumeric characters
SELECT REGEXP_REPLACE('Ab#12@Cd!', '[^A-Za-z0-9]', '') FROM dual; -- Ab12Cd

-- collapse multiple spaces into one
SELECT REGEXP_REPLACE('a   b   c', '{2,}', ' ') FROM dual; -- 'a b c'

-- reformat date DD-MM-YYYY -> YYYY/MM/DD using backreferences
SELECT REGEXP_REPLACE('07-08-2026',
    '(\d{2})-(\d{2})-(\d{4})', '\3/\2/\1') FROM dual; -- 2026/08/07

-- mask all but last 4 digits of a card number
SELECT REGEXP_REPLACE('1234567812345678', '\d(?:=\d{4})', '*') FROM dual;
```

6. REGEXP_COUNT & REGEXP_INSTR

```
SELECT REGEXP_COUNT('a,b,c,d', ',') FROM dual; -- 3
SELECT REGEXP_COUNT('aXbXc', 'X', 1, 'i') FROM dual; -- 2 (case-insensitive)
SELECT REGEXP_INSTR('abc123', '[0-9]') FROM dual; -- 4 (pos of first digit)
```

7. Match Parameter Flags

Flag	Meaning
'i'	Case-insensitive
'c'	Case-sensitive (default)
'm'	Multi-line: ^/\$ match line breaks
'n'	Dot . matches newline
'x'	Ignore whitespace in the pattern

Practical tips

- Common validations: 4-digit code `^\d{4}$`, alphanumeric `^[A-Za-z0-9]+$`, phone, email.
- For Arabic/Unicode text, avoid `[A-Za-z]` (it excludes Arabic) — use `^[[:punct:]][[:space:]][[:digit:]]` or explicit ranges.
- Backreferences `\1 \2 \3` in the replacement string reuse captured groups.
- REGEXP functions are slower than plain `LIKE/SUBSTR` — prefer simple functions when a regex isn't needed.
- The 6th argument of `REGEXP_SUBSTR` selects a capture group — very handy for parsing.